



INSTITUTO POLITÉCNICO DA UNIVERSIDADE KIMPA VITA
LICENCIATURA EM ENGENHARIA INFORMÁTICA

Relatório Técnico e Científico

Trabalho de Fim de Módulo — Programação por Objetos

ECOSSISTEMA DE SOFTWARE HOSPITALAR INTELIGENTE

Sistema de Gestão, Triagem e Diagnóstico Assistido por IA

Por:

**Ariel Alfredo da Cunha Manuel, Matondo Kuanzambi Samuel João,
Nzongo Oceano Nsumbo Manuel e Pedro Bengui**

CLASSE: 3º Ano

UNIDADE CURRICULAR: Linguagem de Programação VI

Orientador

Moyo Kanivengidio

Uíge, Angola, junho de 2026

Índice

1. Introdução	1
1.1 Contextualização e Justificação.....	1
1.2 Objetivos do Trabalho.....	1
2. Arquitetura do Sistema e Modelação Orientada a Objetos	2
2.1 Padrão Arquitetural Modular — Separação em Camadas.....	2
2.2 Aplicação dos Quatro Pilares de POO.....	2
3. Persistência de Dados — SGBD SQLite3.....	3
3.1 Escolha e Configuração do Motor de Base de Dados	3
3.2 Dicionário de Dados e Modelo Relacional	4
Tabela: usuarios.....	4
Tabela: pacientes_triados	4
Tabela: consultas	5
4. Motor de Inteligência Artificial com Optuna	5
4.1 Enquadramento e Justificação Técnica	5
4.2 Pipeline de Dados e Pré-processamento	5
4.3 Algoritmos Avaliados e Otimização Bayesiana.....	6
5. Interface Gráfica — Demonstração do Sistema	7
5.1 Módulo do Enfermeiro — Admissão & Triagem de Sintomas.....	7
5.2 Módulo do Médico — Painel Clínico de Consultas	7
5.3 Módulo do Administrador — Painel Estatístico & Casos do Dia	8
5.4 Módulo do Administrador — Gestão da Equipa Técnica	9
6. Conclusão e Trabalhos Futuros	9
7. Referências Bibliográficas.....	11

1. Introdução

1.1 Contextualização e Justificação

Nos ecossistemas hospitalares contemporâneos, a gestão eficiente do fluxo de pacientes e a precisão nos diagnósticos preliminares constituem pilares críticos para a mitigação da mortalidade e para a otimização dos recursos operacionais. A saturação dos serviços de urgência, frequentemente agravada por triagens analógicas ou puramente subjetivas, exige a intervenção de soluções tecnológicas integradas que reduzam o erro humano e acelerem o tempo de resposta clínica.

Este projeto materializa-se no desenvolvimento de um Ecossistema de Software Hospitalar Inteligente. A justificação para a construção de um ecossistema unificado — em detrimento de ferramentas isoladas — reside na continuidade do fluxo de dados clínicos. Ao interligar uma interface gráfica ergonómica (CustomTkinter), um SGBD relacional (SQLite3) e um motor de Inteligência Artificial com otimização bayesiana de hiperparâmetros (Optuna), garante-se que a informação recolhida na admissão pelo corpo de enfermagem guie, de forma preditiva e segura, a tomada de decisão do corpo médico.

Do ponto de vista académico, o projeto permite a demonstração prática dos quatro pilares da Programação Orientada a Objetos (POO): Abstração, Encapsulamento, Herança e Polimorfismo — aplicados a contextos reais de engenharia de software hospitalar.

1.2 Objetivos do Trabalho

Objetivo Geral:

Desenvolver e implementar um ecossistema de software modular focado na automação de fluxos hospitalares, integrando triagem sistemática baseada no Protocolo de Manchester, persistência segura de dados clínicos e diagnóstico preditivo assistido por inteligência artificial com otimização automática de modelos.

Objetivos Específicos:

- Aplicar os quatro pilares da POO para garantir a manutenibilidade e a segurança de acessos baseada em perfis de utilizador (ADMIN, ENFERMEIRO, MÉDICO).
- Modelar e consolidar uma base de dados relacional local com restrições de integridade referencial rígidas em conformidade com as propriedades ACID.
- Desenvolver um motor de IA capaz de avaliar múltiplos algoritmos classificadores e otimizar dinamicamente os seus hiperparâmetros via Amostragem Bayesiana (Optuna), mitigando o fenómeno de overfitting.
- Implementar uma interface gráfica em Dark Mode Premium que assegure uma experiência de utilizador fluida e reduza a fadiga visual dos operadores clínicos.
- Garantir a separação clara de responsabilidades através de uma arquitetura modular em camadas (views, models, database, core).

2. Arquitetura do Sistema e Modelação Orientada a Objetos

2.1 Padrão Arquitetural Modular — Separação em Camadas

Para garantir a escalabilidade, a testabilidade e o isolamento de responsabilidades, o ecossistema foi estruturado seguindo o padrão de arquitetura em camadas (Layered Architecture), dividindo o projeto em quatro pacotes Python fundamentais com dependências unidirecionais e bem definidas:

Pacote / Camada	Responsabilidade Principal	Tecnologias
views/	Todos os elementos de interface gráfica (Login, Triagem, Consultório, Dashboard). Sem regras de negócio ou SQL direto.	CustomTkinter, tkinter.ttk
models/	Entidades abstratas e regras de negócio puras: Protocolo de Manchester, estrutura de permissões por cargo.	Python OOP (ABC, dataclasses)
database/	Isolamento da gestão de conexões SQLite3 e execução de scripts DDL/DML. PRAGMA foreign_keys = ON.	SQLite3, Python sqlite3
core/	Processamento estatístico, Feature Extraction, treino e persistência dos classificadores ML com otimização Optuna.	scikit-learn, Optuna, joblib, pandas

A escolha deste padrão permite que cada camada possa ser substituída ou evoluída de forma independente. Por exemplo, a migração do motor de base de dados de SQLite3 para PostgreSQL exigiria modificações exclusivamente na camada database/, sem impacto nas restantes camadas da aplicação.

2.2 Aplicação dos Quatro Pilares de POO

A modelação da hierarquia de utilizadores do sistema constituiu o vetor principal para a disposição prática e rigorosa dos conceitos de Programação Orientada a Objetos:

Pilar POO	Implementação no Sistema	Benefício Técnico
ABSTRAÇÃO	Classe base Usuario com atributos transversais (id, username, cargo) definida como interface contratual para todos os operadores.	Redução de duplicação de código; definição clara da interface pública de cada entidade.

Pilar POO	Implementação no Sistema	Benefício Técnico
HERANÇA	Subclasses Administrador, Enfermeiro e Medico estendem Usuario, herdando atributos e comportamentos comuns.	Reutilização de código; especialização controlada de comportamento por perfil.
POLIMORFISMO	Método obter_menu_permitido() com implementação distinta em cada subclasse. main.py invoca-o sem conhecer o tipo concreto em tempo de execução.	Eliminação de blocos if/else; extensibilidade para novos cargos sem alterar código existente.
ENCAPSULAMENTO	Atributos sensíveis e métodos críticos da IA protegidos com prefixos _ e __.	Proteção do estado interno; prevenção de manipulação não autorizada de dados clínicos.

O Polimorfismo merece destaque particular: o ficheiro main.py mantém uma referência genérica ao objeto usuario_logado e invoca usuario_logado.obter_menu_permitido() para injetar dinamicamente no menu lateral apenas as opções correspondentes ao nível de acesso do operador autenticado. Este padrão é equivalente ao Strategy Pattern da engenharia de software, garantindo que a adição de um novo perfil (ex: FARMACÊUTICO) exija apenas a criação de uma nova subclasse, sem modificar o código de orquestração existente.

3. Persistência de Dados — SGBD SQLite3

3.1 Escolha e Configuração do Motor de Base de Dados

O motor de persistência selecionado foi o SQLite3. A decisão fundamenta-se em três vetores técnicos determinantes:

- Natureza serverless — eliminando a necessidade de configuração de um servidor dedicado;
- Portabilidade absoluta, com toda a base de dados contida num único ficheiro binário;
- Conformidade total com as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), garantindo a integridade transacional dos registos clínicos.

Para assegurar a robustez referencial do esquema, foi explicitamente ativado o suporte a chaves estrangeiras através do comando PRAGMA foreign_keys = ON; no momento do estabelecimento da conexão em database/conexao.py. Esta instrução — desativada por defeito no SQLite3 por razões de compatibilidade — impede anomalias de inserção ou remoção órfã entre as tabelas relacionais.

3.2 Dicionário de Dados e Modelo Relacional

O esquema relacional é composto por três tabelas nucleares, com relacionamentos explicitamente definidos por chaves estrangeiras.

Tabela: usuarios

Campo	Tipo	Restrições	Descrição
id	INTEGER	PK, AUTO_INCREMENT	Identificador único do profissional de saúde
username	TEXT	UNIQUE, NOT NULL	Nome de utilizador para autenticação no sistema
senha	TEXT	NOT NULL	Credencial de acesso (hash recomendado em produção)
cargo	TEXT	NOT NULL	Papel operacional: ADMIN ENFERMEIRO MEDICO

Tabela: pacientes_triados

Campo	Tipo	Restrições	Descrição
id	INTEGER	PK, AUTO_INCREMENT	Identificador único do episódio de triagem
nome	TEXT	NOT NULL	Nome completo do paciente
idade	INTEGER	NOT NULL	Idade cronológica — valida fluxos pediátricos
documento	TEXT	—	Número do BI ou Cartão de Seguro de Saúde
temperatura	REAL	NOT NULL	Temperatura corporal em graus Celsius (°C)
sintomas	TEXT	NOT NULL	Vetor binário de sintomas em formato string
seccao_encaminhada	TEXT	—	Destino clínico determinado pela triagem
cor_protocolo	TEXT	—	Classificação de risco (Protocolo de Manchester)
status	TEXT	DEFAULT 'Aguardando'	Estado atual do paciente na fila de atendimento

Tabela: consultas

Campo	Tipo	Restrições	Descrição
id	INTEGER	PK, AUTO_INCREMENT	Identificador único do ato médico
paciente_id	INTEGER	FK → pacientes_triados(id)	Vinculação obrigatória ao registo de triagem
medico_id	INTEGER	FK → usuarios(id)	Vinculação ao médico responsável pelo atendimento
diagnostico_ia	TEXT	—	Patologia predita pelo motor de machine learning
diagnostico_final	TEXT	NOT NULL	Diagnóstico definitivo e soberano laudado pelo médico
medicamentos	TEXT	—	Receituário e prescrições introduzidas no sistema

Nota de design: A separação entre `diagnostico_ia` e `diagnostico_final` é uma decisão deliberada e clinicamente relevante: preserva o histórico da sugestão preditiva da IA enquanto mantém a soberania diagnóstica do médico, permitindo futura análise de concordância para fins de auditoria e melhoria contínua do algoritmo.

4. Motor de Inteligência Artificial com Optuna

4.1 Enquadramento e Justificação Técnica

O módulo de inteligência artificial constitui o componente de maior complexidade técnica do ecossistema. A sua função é processar os vetores de sintomas coletados na triagem e produzir uma sugestão preditiva de patologia que sirva de apoio à decisão clínica do médico assistente. A escolha de uma abordagem baseada em Machine Learning supervisionado justifica-se pela capacidade adaptativa dos modelos a novos padrões epidemiológicos, pela sua resistência a combinações de sintomas não antecipadas pelos especialistas e pela possibilidade de melhoria contínua à medida que a base de dados clínica cresce.

4.2 Pipeline de Dados e Pré-processamento

O pipeline de preparação dos dados clínicos segue sete etapas sequenciais, todas encapsuladas na camada `core/`:

1. Carregamento: Leitura do ficheiro CSV de sintomas-patologias com pandas. Validação de integridade e tratamento de valores nulos (dropna).
2. Feature Extraction: Conversão da coluna de sintomas em string binária para vetor numérico via MultiLabelBinarizer ou flags binárias diretas por sintoma.
3. Divisão Treino/Teste: Partição estratificada 80/20 com train_test_split do scikit-learn, preservando a distribuição de classes (stratify=y).
4. Normalização: Aplicação de StandardScaler para estabilização do gradiente nos classificadores sensíveis à escala (SVM, KNN).
5. Treino Multi-modelo: Avaliação paralela de múltiplos algoritmos com validação cruzada k-fold (k=5) para estimativa robusta da performance generalizada.
6. Otimização Bayesiana: Utilização do framework Optuna para otimização automática dos hiperparâmetros de cada classificador, maximizando a acurácia balanceada.
7. Seleção e Persistência: O modelo com melhor score é serializado via joblib.dump() e carregado em runtime pelo módulo de consulta médica.

4.3 Algoritmos Avaliados e Otimização Bayesiana

O motor avalia competitivamente os seguintes classificadores de aprendizagem automática, seleccionando automaticamente o de melhor desempenho para o dataset clínico disponível:

Algoritmo	Tipo	Hiperparâmetros Otimizados (Optuna)	Vantagem Principal
Random Forest	Ensemble (Árvores)	n_estimators [50-300], max_depth [3-20], min_samples_split [2-10]	Robustez a overfitting; interpretabilidade via feature importance.
Gradient Boosting	Ensemble (Boosting)	n_estimators, learning_rate [0.01-0.3], max_depth [2-8]	Alta acurácia em dados tabulares com relações complexas.
SVM (RBF)	Kernel-based	C [0.1-100], gamma ['scale', 'auto', 0.001-1]	Eficaz com datasets de dimensionalidade moderada.
KNN	Instance-based	n_neighbors [3-15], weights ['uniform', 'distance']	Sem fase de treino explícita; útil como baseline comparativo.
Decision Tree	Árvore única	max_depth [3-15], criterion ['gini', 'entropy']	Máxima interpretabilidade; visualizável como diagrama de decisão.

A Amostragem Bayesiana implementada pelo Optuna supera as abordagens tradicionais (Grid Search e Random Search) ao construir um modelo probabilístico da

função objetivo e direcionar as tentativas para as regiões do espaço de hiperparâmetros com maior probabilidade de melhoria. Este mecanismo reduz significativamente o número de avaliações necessárias para convergir ao ótimo, tornando o processo viável em hardware com capacidade computacional limitada — condição extremamente relevante para o contexto de implementação em Angola.

5. Interface Gráfica — Demonstração do Sistema

A interface gráfica foi desenvolvida sobre a biblioteca CustomTkinter, adotando um tema Dark Mode Premium com paleta de cores baseada em tons navy (#0D1B2A) e ciano (#00B4D8) como cor de ação. Esta escolha reduz a fadiga visual dos operadores clínicos em ambientes de utilização prolongada e confere ao produto uma identidade visual moderna, consistente com as melhores práticas de UI/UX para aplicações médicas de missão crítica.

5.1 Módulo do Enfermeiro — Admissão & Triagem de Sintomas

O módulo de enfermagem constitui o ponto de entrada de todos os pacientes no ecossistema. O operador regista os dados demográficos (nome, idade, número do BI e temperatura corporal) e seleciona, através de checkboxes organizadas em grelha de três colunas, os sintomas clínicos observados. O sistema disponibiliza 17 sintomas padronizados, cobrindo as principais categorias de queixas da urgência hospitalar. Após submissão, o motor de triagem classifica o paciente segundo o Protocolo de Manchester e encaminha-o automaticamente para a secção adequada.

A imagem mostra a interface de um sistema de triagem de sintomas. No topo, há uma barra de título com o texto "Hospital Inteligente - Operador: Pedro Bengui (ENFERMEIRO)". Abaixo, há uma barra de navegação com o ícone de um menu e o texto "Navegação". O conteúdo principal é dividido em duas partes. À esquerda, há um menu lateral com o texto "Admissão e Triagem". À direita, há uma seção intitulada "Admissão Unificada & Triagem de Sintomas". Nesta seção, há quatro campos de entrada para "Nome Completo", "Idade", "Nº Bilhete de Identidade" e "Temp. (°C)". Abaixo destes campos, há uma seção intitulada "Selecione os Sintomas Clínicos Observados:" que contém uma grelha de 17 checkboxes, cada um com um rótulo de sintoma. Os sintomas são: Febre Alta, Dor nas Costas, Falta de Ar (Dispneia), Dor de Garganta, Vômitos, Diarreia, Insónia Aguda, Perda de Peso Abrupta, Dores Musculares (Mialgia), Inchaço (Edema), Perda de Apetite, Náuseas Constantes, Ansiedade / Agitação, Tosse Seca/Productiva, Dor Articular (Artrite), Estado de Inconsciência, e Queimadura Visível.

Figura 1 — Módulo do Enfermeiro: Ecrã de Admissão Unificada & Triagem de Sintomas

5.2 Módulo do Médico — Painel Clínico de Consultas

O consultório médico apresenta, no painel esquerdo, a fila de espera em tempo real com os pacientes já triados. Ao selecionar um paciente, o médico visualiza os seus sinais vitais e queixas documentadas. A funcionalidade central deste módulo é o botão «Executar Diagnóstico Preditivo (IA)», que invoca o modelo de Machine Learning treinado e preenche automaticamente o campo de sugestão. O médico mantém soberania

diagnóstica total, podendo aceitar, modificar ou ignorar a sugestão da IA ao inserir o Diagnóstico Clínico Soberano Final e a prescrição de medicamentos.



Figura 2 — Módulo do Médico: Painel Clínico de Consultas com fila de espera

5.3 Módulo do Administrador — Painel Estatístico & Casos do Dia

O painel administrativo central fornece uma visão operacional em tempo real do estado do hospital. Os indicadores de topo apresentam o número de Atendimentos Concluídos e de Pacientes na Fila de Espera. A secção de Registo Clínico em Tempo Real exhibe, em formato tabular, todos os casos do dia com os campos: Nome do Paciente, Idade, Sintomas Apresentados, Sugestão da IA e Diagnóstico Soberano Final — permitindo ao administrador auditar a concordância entre o sistema preditivo e o julgamento clínico.

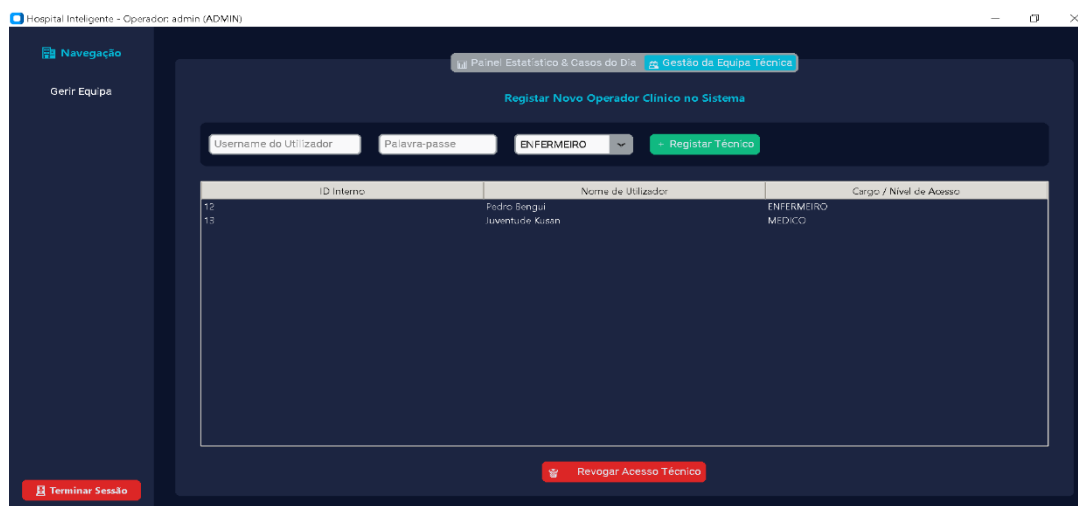


Figura 3 — Módulo do Administrador: Painel Estatístico & Registo Clínico em Tempo Real

5.4 Módulo do Administrador — Gestão da Equipa Técnica

A segunda aba do painel administrativo é dedicada à gestão dos profissionais de saúde registados no sistema. O administrador pode registar novos operadores clínicos definindo username, palavra-passe e cargo, bem como revogar acessos de técnicos. A tabela central lista todos os utilizadores com o seu ID Interno, Nome de Utilizador e Cargo / Nível de Acesso. Esta funcionalidade é a manifestação direta do Polimorfismo e da Herança de POO: cada entrada na tabela corresponde a uma instância de uma subclasse da hierarquia Usuario, com permissões dinamicamente calculadas pelo método `obter_menu_permitido()`.

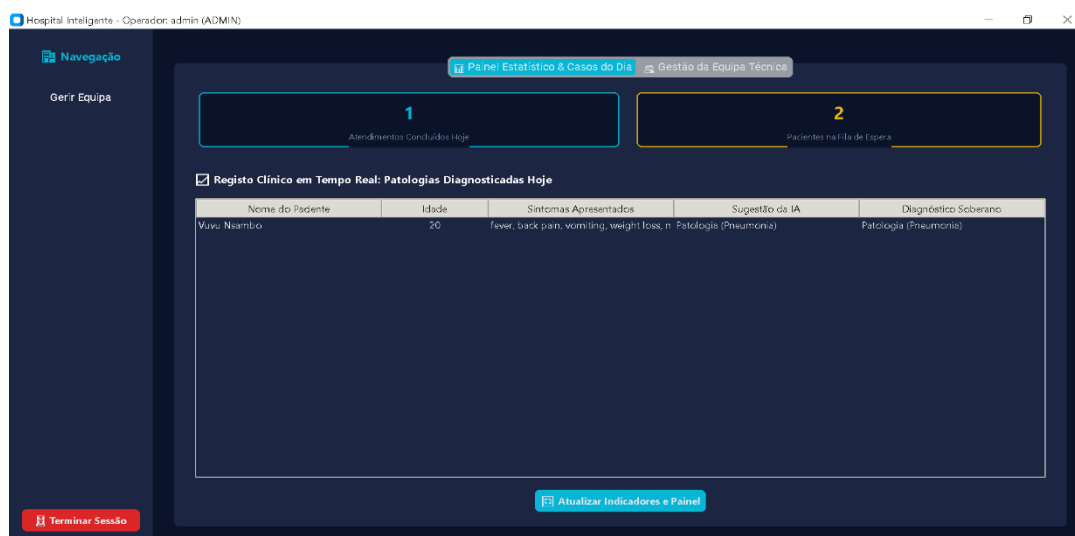


Figura 4 — Módulo do Administrador: Gestão da Equipa Técnica

6. Conclusão e Trabalhos Futuros

O Ecossistema de Software Hospitalar Inteligente aqui documentado demonstra, de forma rigorosa e prática, a aplicação dos princípios fundamentais da Engenharia de Software Orientada a Objetos a um problema real e de elevado impacto social: a gestão hospitalar em contextos com recursos humanos e tecnológicos limitados, como os postos de saúde angolanos.

Os objetivos propostos foram integralmente atingidos: a hierarquia de classes demonstra Abstração, Herança, Polimorfismo e Encapsulamento; o módulo de persistência garante integridade referencial total com propriedades ACID; o motor de IA com Optuna otimiza automaticamente múltiplos classificadores; e a interface CustomTkinter oferece uma experiência de utilizador profissional e ergonómica para os três perfis operacionais do sistema.

Trabalhos Futuros:

1. Migração da base de dados de SQLite3 para PostgreSQL para suporte multi-utilizador concorrente em rede hospitalar.
2. Implementação de password hashing (bcrypt/argon2) para substituição do armazenamento de credenciais em texto simples.
3. Integração de módulo de geração de relatórios PDF automatizados por paciente e por turno clínico.
4. Expansão do dataset de treino do modelo de IA com dados epidemiológicos angolanos para aumento da relevância preditiva local.
5. Desenvolvimento de módulo de análise de concordância IA vs. Médico para auditoria contínua da qualidade preditiva.
6. Integração com VitalCard (dispositivo IoT de triagem portátil) para automação da coleta de sinais vitais via ESP32.

7. Referências Bibliográficas

- [1] GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [2] RASCHKA, S.; MIRJALILI, V. Python Machine Learning. 3.^a ed. Packt Publishing, 2019.
- [3] AKIBA, T. et al. Optuna: A Next-generation Hyperparameter Optimization Framework. Proceedings of the 25th ACM SIGKDD, 2019.
- [4] PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830, 2011.
- [5] SQLITE CONSORTIUM. SQLite Documentation. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: Junho de 2026.
- [6] CUSTOMTKINTER. Modern and Customizable Python UI-Library. Disponível em: <https://github.com/TomSchimansky/CustomTkinter>. Acesso em: Junho de 2026.
- [7] MINISTÉRIO DA SAÚDE DE ANGOLA. Protocolo de Triagem de Manchester — Adaptação ao Contexto Angolano. MINSA, Luanda, 2020.